



1. (20 نمره)

الف- (10 نمره) تابعی بنویسید که ابعاد یک ماتریس مربعی N و خود ماتریس $matrix$ را دریافت کند. این تابع باید مجموع عناصر موجود در مثلث بالایی (شامل قطر اصلی و عناصر بالای آن) را محاسبه کرده و برگرداند. (نکته: فقط مجاز به استفاده از اشاره گر هستید و نباید از اندیس‌گذاری آرایه $[]$ استفاده کنید)

```
int sumUpperTriangle(int N, int matrix[N][N]) {  
  
}
```

ب- (10 نمره) تابعی بنویسید که ابعاد یک ماتریس مربعی N و خود ماتریس $matrix$ را دریافت کند. این تابع باید بررسی کند آیا ماتریس ورودی یک ماتریس همانی است یا خیر. اگر بود 1 و در غیر این صورت 0 برگرداند. (فقط مجاز به استفاده از اشاره گر هستید)
(راهنمایی: ماتریس همانی ماتریسی است که تمام عناصر روی قطر اصلی آن 1 و بقیه عناصر 0 باشند)

```
int isIdentityMatrix(int N, int matrix[N][N]) {  
  
}
```

2. (20 نمره) تابعی به نام `deleteElement` بنویسید. این تابع سه پارامتر را به صورت اشاره‌گر دریافت می‌کند تا بتواند عملیات زیر را بر روی آرایه ورودی انجام دهد:

جستجو: عنصر با مقدار $*target$ را در آرایه‌ای که از آدرس $addr$ شروع می‌شود، پیدا کند.
حذف و شیفت: اگر عنصر پیدا شد، اولین رخداد آن را حذف کند؛ یعنی تمام عناصر موجود در خانه‌های بعدی باید یک خانه به سمت چپ جابجا (شیفت) شوند.
به‌روزرسانی سایز: در صورت موفقیت‌آمیز بودن حذف، مقدار متغیر $*size$ (که نشان‌دهنده تعداد عناصر موجود در آرایه است) باید یک واحد کاهش یابد.

```
void deleteElement(int *addr, int *target, int *size) {  
  
}
```

3. (10 نمره) یک مزیت از $pass\ by\ value$ و یک مزیت از $pass\ by\ reference$ را بیان کنید.
یک مثال از شرایطی بزنید که $pass\ by\ reference$ نسبت به $pass\ by\ value$ بهتر است (نیاز به نوشتن کد نیست).

4. امتیازی** (15 نمره) فرض کنید arr یک آرایه با اعداد صحیح به طول n است که به دو زیر بخش تقسیم شده است به طوری که هر دو زیر بخش مرتب شده هستند. فرض کنید زیر بخش اول از اندیس 0 تا اندیس r است و زیر بخش دوم از اندیس $r + 1$ تا اندیس $n - 1$ است. تابع زیر را به گونه‌ای تکمیل کنید تا این آرایه arr به همراه دو پارامتر دیگر r و n را بگیرد و این آرایه را به طور کامل مرتب کند. همچنین برای حل این سوال از اشاره گر ها استفاده کنید.

```
void mergeSubarrays(int *arr, int r, int n) {  
  
}
```

راهنمایی: از اندیس $r + 1$ شروع کنید و عناصر را تک تک به زیر بخش اول اضافه کنید، طوری که هر دو زیر بخش، به صورت مرتب شده باقی بمانند.

5. امتیازی** (25 نمره) ابتدا توضیح دهید که فرق بین `int (*arr1)[5]` و `int *arr2[5]` چیست، سپس با توضیح خط به خط خطوط 1 تا 12 (مشخص شده با کامنت) ، خروجی برنامه را نمایش دهید.

راهنمایی: وقتی میگوییم `a` یک آرایه به طول `n` است، یعنی `a` به بخشی از حافظه اشاره می‌کند که از `n` خانه متوالی تشکیل شده و پشت سر هم قرار گرفته اند. برای دسترسی به آدرس عناصر، از نام این متغیر به همراه عملگر های جمع و تفریق استفاده میشود. مثلاً:

`a` یعنی آدرس اولین خانه

`a + 3` یعنی آدرس چهارمین خانه

اگر `b` یک اشاره گر به پنجمین خانه حافظه آرایه `a` باشد، `b - 2` یعنی آدرس سومین خانه آرایه `a` و برای دسترسی به مقدار عناصر، میتوانیم از `[ ]` یا `*` استفاده کنیم. مثلاً:

`a[2]` یعنی مقدار سومین خانه

`*(a + 1)` یعنی مقدار دومین خانه

مثال از آرایه تک بعدی و دو بعدی:

$$a + i \equiv \&a[i] , *(a + i) + j \equiv a[i] + j \equiv \&a[i][j]$$

$$*(a + i) \equiv a[i] , (*(a + i) + j) \equiv *(a[i] + j) \equiv a[i][j]$$

```
#include <stdio.h>

void function(int (*arr1)[5], int *arr2[5]) {
    int *p = (*arr1) + 1;           // 3
    int *q = arr2[2] - 1;           // 4

    (*arr1)[2] = arr2[0][1] + *p;    // 5
    arr2[1] = (*arr1) + 3;           // 6

    p = *arr1 + 3;                   // 7
    *arr2[1] += q[1];                // 8

    *(arr2 + 3) = arr2[1] - 3;       // 9
    q = arr2[4];                     // 10

    *(arr2[3] + 2) = arr2[1][0] + (*arr1)[0] + q[1]; // 11
    *p = arr2[3][2] - (*(arr2 - 1)); // 12
}
```

```

int main() {
    int x[5] = {3, 0, 1, 7, 6};
    int y[5] = {9, 4, 8, 5, 2};

    int (*arr1)[5] = &x; // 1
    int *arr2[5] = {y + 3, &x[2], &y[4], x + 1, y}; // 2

    function(arr1, arr2);

    for(int i = 0; i < 5; i++){
        printf("%d\t", x[i]);
    }
    printf("\n");

    for(int i = 0; i < 5; i++){
        printf("%d\t", *(y + i));
    }
    printf("\n");

    for(int i = 0; i < 5; i++){
        printf("%d\t", *(arr2[i]));
    }
    printf("\n");

    return 0;
}

```

⚠ در تحویل تمرین، حتما به نکات زیر توجه نمایید:

❖ هیچ گونه شباهتی در انجام تمرین بین افراد مختلف پذیرفته نمی‌شود. در صورت کشف هرگونه تقلب مطابق قوانین درس با افراد خاطی برخورد خواهد شد.
○ پیشنهاد می‌شود از امکان رفع اشکال خصوصی، حداکثر استفاده را نمایید!

❖ پاسخ هر پرسشی که امکان آپلود برای آنها فراهم شده است را به صورت یک فایل pdf. و با نام `HW6_<StdID>` آپلود نمایید. (که StdID شماره دانشجویی شماست. به عنوان مثال اگر شماره دانشجویی کسی 810100000 باشد، فایل بارگذاری شده او باید به نام HW6_810100000 باشد)

❖ مهلت تحویل تمرین تا 1404/10/4 می‌باشد.
○ تمرین تنها در سامانه Elearn دریافت خواهد شد و از ایمیل آن خودداری نمایید.
○ مهلت تحویل تمرین، تمدید نخواهد شد؛ در منظم بودن تحویل، کوشا باشید.

❖ پس از مطالعه کامل تمرین، در صورت هرگونه ابهام با [آرش قربانخانی](#) یا [امین غلامی](#) در ارتباط باشید.
○ موضوع ایمیل را `HW6 ICSP` وارد نمایید.